

Visual HPC for Engineering Students

(VHPCE)

An Interactive Performance Laboratory for Teaching
High-Performance Computing through Experimentation

Nikos Vilardos
nikosvil@gmail.com

June 2026

Abstract

Teaching High-Performance Computing (HPC) to engineering students presents a persistent challenge: the gap between textbook theory and the reality of hardware behavior. Students learn Amdahl's law and cache hierarchies in lecture, but rarely get to observe these phenomena on real silicon. VHPCE bridges this gap with an interactive, browser-based performance laboratory where students edit code, run experiments, watch hardware behavior animate in real time, and receive plain-English explanations of why performance changes. The platform covers OpenMP, MPI, OpenACC, and CUDA through sixteen flagship experiments, a 214-entry command reference, engineering domain simulation labs, a live code playground with Docker-sandboxed execution, and a gamification layer. A central architectural idea—the *ProfileResult* seam—lets every experiment run in two interchangeable modes: a physics-based model (offline, instant) and measured runs on the student's own CPU/GPU via Docker. This paper presents the system architecture, pedagogical design, implementation details, and deployment model.

Keywords: HPC education, parallel computing, interactive visualization, performance modeling, OpenMP, MPI, CUDA, Docker sandboxing

Table of Contents

- 1. Introduction 3
- 2. System Architecture 4
- 3. Pedagogical Design 6
- 4. The Experiment Engine 8
- 5. Interactive Components 10
- 6. Execution Backend 12
- 7. AI-Powered Explanation Layer 14
- 8. Deployment Model 15
- 9. Future Work 16
- 10. Conclusion 17

1. Introduction

High-Performance Computing underpins modern science and engineering—from climate modeling and computational fluid dynamics to molecular simulation and machine learning training. Yet teaching HPC remains difficult. The core challenge is not syntax (students can learn `#pragma omp parallel` for in an afternoon) but **performance intuition**: understanding *why* adding more threads sometimes makes code slower, why memory access patterns matter more than instruction counts, and why the gap between a textbook formula and real hardware can be enormous.

Existing approaches fall into two camps. Lecture-based courses cover Amdahl's law, cache hierarchies, and communication overhead in theory, but students never observe these phenomena on real hardware. Lab-based courses run code on clusters, but the turnaround time between editing and insight is long, the infrastructure is fragile, and students often cannot distinguish a code bug from a hardware effect.

VHPCE—**Visual HPC for Engineering Students**—is an interactive, browser-based performance laboratory that attacks this gap head-on. Its design philosophy can be summarized in one phrase: **performance intuition over syntax memorization**. Students don't just write parallel code; they *predict* its behavior, *observe* what actually happens (via animated visualizations and scaling charts), and *read* a deterministic, plain-English diagnosis explaining the discrepancy. The platform covers the full HPC stack—OpenMP, MPI, OpenACC, and CUDA—through a unified interface.

1.1 Key Contributions

- **The ProfileResult seam.** A single data contract that decouples data producers (physics models and real hardware measurements) from consumers (visualizations, charts, and explanations). This lets the platform ship a fully functional offline experience on day one and upgrade to real measurements later without changing any downstream code.
- **Predict-before-you-run pedagogy.** Every experiment asks the student to guess the speedup before running, grounding the learning cycle in active recall rather than passive observation.
- **Model and Measured duality.** Every experiment supports two interchangeable data sources: an instant physics model (runs in the browser, no backend needed) and measured runs on the student's own CPU or GPU via Docker containers.
- **Sixteen flagship experiments** covering the most common and counter-intuitive performance bottlenecks in parallel programming, each with 2D/3D visualization, scaling charts, and deterministic diagnostics.
- **A locked-down code playground** where students compile and run arbitrary OpenMP C in Docker containers with network isolation, capability dropping, memory/PID limits, and read-only filesystems.
- **A dual AI explanation layer**—always-on deterministic rules grounded in the physics models, plus an optional LLM panel for open-ended follow-up questions.

2. System Architecture

VHPCE is built as a TypeScript/Python monorepo with a clear separation between data production, data contracts, and data consumption. The central architectural insight is that a single schema—the **ProfileResult**—serves as the interface between everything that generates performance data and everything that renders or explains it.

2.1 The ProfileResult Seam

The `ProfileResult` is a JSON schema (with generated TypeScript types, Zod validators, and a mirrored Pydantic model) that describes the output of any performance experiment. It includes timing data, scaling sweeps, roofline parameters, hardware counters, thread timelines, MPI communication events, GPU occupancy metrics, and pre-computed diagnostic codes. The schema is versioned and self-describing: every numeric field carries its unit in the field name (`_gbps`, `_ns`, `_gflops`).

Two producers feed this schema. **Producer A** (`@vhpce/perf-models`) runs entirely in the browser: it computes physics-based performance curves using Amdahl's law, roofline analysis, bandwidth saturation models, and coherence-cost formulas. **Producer B** (the FastAPI gateway) compiles and runs real OpenMP, MPI, and CUDA kernels in Docker containers on the student's own hardware and returns measured `ProfileResults`. Because the schema is producer-agnostic, all downstream consumers—visualizations, charts, explanations—work identically with either source.

2.2 Monorepo Structure

The repository uses pnpm workspaces with Turborepo for task graph caching. The key packages are:

Package	Role
apps/web	Next.js 16 / React 19 / Tailwind v4 web application
packages/profile-schema	The ProfileResult seam: types, Zod validators, formatters
packages/perf-models	Producer A: physics models + measured data adapters
packages/explain	Deterministic explanation engine (rule-based diagnostics)
packages/viz	D3 charts (scaling, roofline, occupancy) + Canvas 2D scenes
services/api	FastAPI gateway + Redis/Arq job queue (Producer B)
services/runner/experiments	C/CUDA kernels: bench.c, halo.c, occupancy.cu
infra/docker	Dockerfiles and compose for sandboxed execution

2.3 Tech Stack

The frontend is built with **Next.js 16** (App Router) and **React 19** with Tailwind CSS v4 for styling. 3D visualizations use **React Three Fiber** (Three.js) with a WebGL2 fallback. 2D charts use **D3** for scaling curves, roofline plots, and occupancy charts. The code editor is **Monaco** (the VSCode engine). The backend gateway is **FastAPI** (Python 3.12+) with **Redis** for caching and **Arq** for async job queuing. The optional AI panel uses the **Anthropic Claude** SDK with streaming and prompt caching.

3. Pedagogical Design

VHPCE's pedagogical approach is built on three principles: **active prediction** over passive observation, **immediate visual feedback** over deferred analysis, and **progressive depth** from conceptual understanding to hardware-level insight.

3.1 The Learning Journey

The platform defines a guided path through eleven interconnected pages, designed to take a student from "what is parallelism" to "why doesn't my code scale":

- **Introduction** (/intro) — The front door. Explains the model-vs-measured idea and provides a map of every section with estimated time investment.
- **Start Here** (/start) — Three live runs that teach the whole story in five minutes: parallel works, the naïve version doesn't scale (a synchronization bottleneck), and the one-line fix (reduction) does. Each step is a *predict-before-you-run*: students guess the speedup before hitting Run.
- **Learn the Basics** (/learn) — Six animated concept walkthroughs covering fork-join, parallel-for loop splitting, shared vs. private memory, MPI ranks and separate memory, point-to-point communication, and collective operations.
- **Command Reference** (/reference) — A searchable library of ~214 directives across OpenMP, MPI, OpenACC, and CUDA, each with a looping Canvas animation, plain-English summary, C/Fortran toggle, and links to runnable examples.
- **Domain Labs** (/modules) — Engineering simulation mini-labs (Wave/FDTD, N-Body Gravity, GEMM) showing how the same computation maps to serial, OpenMP, MPI, and GPU code.
- **Heat Lab** (/lab) — A 2D heat-equation solver with domain-decomposition overlays and a live convergence plot.
- **Flagship** (/) — Sixteen experiments covering every major parallel performance bottleneck.
- **Compare** (/compare) — The textbook model overlaid on real measurements, highlighting where theory and hardware agree and diverge.
- **Code Playground** (/playground) — Write arbitrary OpenMP C, predict the speedup, compile and run in a sandboxed container.
- **Play** (/play) — Gamified activities: Race (head-to-head kernel comparison), Quiz, Sandbox (Amdahl/Gustafson explorer), Kernel Tuner, and Badges.

3.2 Predict-Before-You-Run

Research in active learning consistently shows that prediction followed by feedback produces stronger retention than observation alone. VHPCE embeds this throughout: in Start Here, the Playground, and the Flagship experiments, students are asked to guess the speedup or identify the bottleneck *before* seeing the result. Correct predictions earn badges and streaks; incorrect ones route to the specific experiment or explanation that addresses the misconception.

3.3 Model vs. Measured

The Model/Measured toggle is not just a convenience—it is itself a teaching tool. The model shows the textbook-clean version of each phenomenon: perfect Amdahl curves, ideal bandwidth saturation, clean

communication walls. The measured run shows reality on the student's machine: noise, oversubscription effects, hardware quirks. The *Compare* page overlays both, making the gap between theory and practice the explicit object of study.

4. The Experiment Engine

The Flagship page hosts sixteen experiments, each built on a documented physics model with no arbitrary constants. Every experiment follows a shared layout: controls (threads/ranks slider, variant toggles), a 2D/3D visualization, a D3 scaling chart, and a structured what/why/how/expected diagnosis.

4.1 Original Nine Experiments

Experiment	Core Lesson	Sweep Axis	Tech
False Sharing	Cache-line coherence ping-pong	Threads	OpenMP
Synchronization	Critical section serialization vs. reduction	Threads	OpenMP
Bandwidth Saturation	DRAM bus saturation + roofline	Threads	OpenMP
Load Imbalance	Static vs. dynamic scheduling	Threads	OpenMP
MPI Halo Exchange	Strong vs. weak scaling, comm fraction	Ranks	MPI
GPU Occupancy	Register pressure, threads/block sweep	Block size	CUDA
GPU Coalescing	Stride sweep, memory transactions	Stride	CUDA
GPU Divergence	Branch-path sweep, warp serialization	Paths	CUDA
GPU Atomics	Concurrent-target sweep, contention	Targets	CUDA

4.2 Seven New Experiments

Seven additional model-only experiments extend coverage to advanced topics:

Experiment	Core Lesson
NUMA Effects	Socket locality, first-touch policy
Cache Hierarchy	L1/L2/L3/DRAM bandwidth cliffs
SIMD Vectorization	SIMD lanes, AoS vs. SoA, AVX
OpenMP Tasks	Irregular task-graph parallelism
GPU Shared Memory	Bank conflicts, 32 banks, padding
MPI Collectives	Broadcast/allreduce/alltoall algorithms
Hybrid MPI+OMP	Ranks $\times$ threads, NUMA sweet spot

4.3 Physics Models

Every curve displayed in Model mode is derived from a documented formula. The key models include:

- **Amdahl's Law:** $S(p) = 1 / ((1 - f) + f/p)$. Drives the speedup ceiling for every experiment. Gustafson's law ($S(p) = p - \alpha(p - 1)$) is used for weak-scaling views.
- **Roofline:** $P_{\text{attainable}}(\text{AI}) = \min(P_{\text{peak}}, \text{AI} \times \text{BW}_{\text{peak}})$. Classifies kernels as memory-bound or compute-bound.

- **False Sharing:** $T(p) = T_{\text{compute}}(p) + T_{\text{coherence}}(p)$, where coherence cost scales as $(p - 1)/p$ for shared cache lines.

- **Bandwidth Saturation:** $BW_{\text{eff}}(p) = BW_{\text{peak}} \times p / (p + p_{\text{half}})$. Models the saturating throughput curve.

- **Load Imbalance:** $T(p) = \max_t(\text{work}_t)$. Static scheduling on a triangular loop yields $I \approx 2$; dynamic drives $I \rightarrow 1$.

All models are parameterized by a named reference machine profile (cores, cache sizes, peak bandwidth, peak compute) so curves are concrete and reproducible. Unit tests assert analytic limits (e.g., $S(1) = 1$, $S(\infty) = 1/(1-f)$).

5. Interactive Components

5.1 Command Reference Library

The Reference page (/reference) is a searchable, filterable library of approximately 214 directives and API calls across six technologies: OpenMP (72 entries), MPI (59), OpenACC (27), CUDA (24), Slurm, and pthreads. The design is **archetype-driven**: 14 reusable Canvas animation archetypes (fork-join, barrier, reduction, scatter/gather, offload, etc.) are defined in code, and each entry maps to an archetype plus parameters. Adding a new entry requires only adding data to a table, not writing animation code.

Each entry includes: a looping Canvas animation showing the directive's behavior, a C/Fortran syntax toggle (derived automatically with manual overrides), a plain-English summary and "good to know" note with hover-glossary terms, related links (cross-technology navigation resets the filter), and a "■ Run in the Playground" link where applicable. An Essentials filter (★) narrows the library to must-knows for beginners, including 11 CUDA essentials.

5.2 Domain Simulation Labs

The Domain Labs hub (/modules) hosts three engineering simulation mini-labs, each with tab-based navigation, a live Canvas animation loop, and domain-decomposition code snippets showing how the same computation maps to serial, OpenMP, MPI, and GPU implementations:

- **Wave Lab (FDTD)** — A 1D finite-difference time-domain wave solver on a 256-cell grid with reflective/absorptive boundary controls. The animation shows wave propagation; code snippets show `omp parallel for`, `MPI_Sendrecv` ghost exchange, and a CUDA kernel.
- **N-Body Gravity** — A 32-particle gravitational simulation using a Barnes–Hut $O(N \log N)$ approximation. Animated particle field with code snippets for OpenMP force loops, MPI particle broadcast, and CUDA thread mapping.
- **Matrix Multiply (GEMM)** — $C = A \times B$ on 8×8 visualization grids. Naïve mode animates the stride- N column access pattern (B column = cache miss); tiled mode shows $NB \times NB$ blocking with cache-resident tiles. An arithmetic intensity readout color-codes the kernel (red < 1 , amber 1–4, green > 4 = ridge point).

The hub also links to the Heat Lab (/lab), a 2D heat-equation solver with domain-decomposition overlays (serial/OpenMP/MPI/GPU) and a live convergence plot showing the stability cliff at $\alpha > 0.25$.

5.3 Code Playground

The Playground (/playground) provides a Monaco code editor where students write arbitrary OpenMP C, predict the speedup, then compile and run a thread-count sweep in a sandboxed Docker container. It ships with eleven worked examples (deep-linkable via `?ex=<id>`). Results include: a scaling chart, plain-language diagnostics that route the student to the specific Flagship experiment explaining their bottleneck, a per-thread-count breakdown explorer, and optional Valgrind cachegrind profiling showing D1/LLd cache miss rates.

5.4 Play Hub

The Play page (/play) is the gamified layer, designed to reinforce learning through engagement:

- **Race** — Two kernel variants run head-to-head; students predict which is faster before seeing the result.
- **Quiz** — A bottleneck identification challenge: given a scaling curve and context, students name the performance pathology.
- **Sandbox** — An interactive explorer for Amdahl's and Gustafson's laws with parameter sliders.
- **Kernel Tuner** — Students adjust GPU launch parameters (threads/block, tile size, registers) and see how they shift occupancy and arithmetic intensity.
- **Badges** — A reward system earned by correct predictions across Start, Playground, and Flagship experiments, with streak tracking.

6. Execution Backend

The execution backend enables Measured mode and the Code Playground. It is entirely optional—Model mode runs fully offline in the browser.

6.1 Gateway Architecture

The gateway (`services/api`) is a FastAPI application fronting a Redis-backed Arq job queue. The web app submits jobs via `POST /api/jobs` and polls results via `GET /api/jobs/{'{'}}id{'}'}`. The Arq worker processes one job at a time (`max_jobs=1`) to ensure clean timing without interference from concurrent workloads. Four job kinds are supported: `bench` (fixed flagship kernels), `code` (arbitrary user code), `mpi` (rank sweeps), and `cuda` (GPU experiments). Fixed-kernel results are cached in Redis (~24h) so repeated requests return instantly.

6.2 Docker Sandboxing

All code execution happens in sibling Docker containers spawned by the worker. The worker mounts the host Docker socket and issues `docker run` commands. Four container images handle the different workloads:

Image	Contents	Security
vhpc-bench	Pre-compiled OpenMP kernels (<code>-O2 -fopenmp -fcommon</code>)	Read-only, no network, cap-drop ALL
vhpc-runner	GCC + Valgrind for arbitrary user code	Read-only, no network, cap-drop ALL, tmpfs /tmp, non-root
vhpc-mpi	OpenMPI + halo exchange kernel	No network (loopback/shm transport), cap-drop ALL
vhpc-cuda	CUDA toolkit + occupancy kernel (<code>nvcc, sm_120</code>)	Read-only + <code>--gpus all</code> , no network, cap-drop ALL

The untrusted code runner (`vhpc-runner`) applies the strictest isolation: `--network none` (no internet), `--cap-drop ALL` (no Linux capabilities), `--security-opt no-new-privileges`, `--read-only filesystem` (except a tmpfs for compilation), 2 GB memory limit, 256 PID limit, and a per-run timeout. Source code is passed via stdin (never written to the host filesystem). The container runs as a non-root user.

6.3 Execution Flow

A typical measured run proceeds as follows: (1) the web app submits a job to the gateway; (2) the gateway enqueues it in Redis via Arq; (3) the worker dequeues the job and spawns the appropriate Docker container; (4) the container runs a thread or rank sweep (best-of-N timing at each point) and prints results as JSON to stdout; (5) the worker captures the output, parses it into a `ProfileResult`, stores it in Redis, and marks the job done; (6) the web app's next poll retrieves the result and renders it through the same charts, metrics, and explanations used for model data.

7. AI-Powered Explanation Layer

VHPCE employs a dual-layer explanation system, summarized as the "Both" design: an always-on deterministic explainer plus an optional live-LLM panel. The two layers cooperate, with the deterministic layer grounding the LLM to prevent hallucination.

7.1 Deterministic Explainer

The deterministic engine (`packages/explain`) is a pure function: `explain(result, context) → Explanation[]`. It runs ordered rule checks over the `ProfileResult`. Each rule has a guard condition and emits a structured finding when triggered, containing: **what** (observed symptom with actual numbers), **why** (the mechanism, naming the HPC concept), **how** (a concrete fix), and **expected gain** (quantified from the model). Findings are deterministic, reproducible, and testable. They appear under every experiment as the default explanation—no API key required.

7.2 LLM Panel

The optional "Ask the AI" panel uses the Anthropic Claude SDK (streaming) to answer open-ended follow-up questions. The key design decision is **grounding**: the LLM receives the deterministic findings as authoritative ground truth alongside the `ProfileResult` summary and the student's question. The system prompt instructs the model to explain and extend the deterministic findings, not to invent competing numbers. This is the primary hallucination control.

The system prompt is large and static, making it an ideal candidate for Anthropic's prompt caching—reducing cost on repeated queries. API keys are provided via a server-side environment variable for hosted deployments, or per-session in the browser (kept per-tab, never persisted). Without any key, the panel shows a notice and the deterministic explanations remain fully active.

7.3 Comparison

	Deterministic	LLM Panel
Availability	Always, offline, free	Requires API key
Output	Structured, exact numbers	Conversational, handles follow-ups
Risk	None (computed)	Hallucination (mitigated by grounding)
Role	Default explanation for every experiment	"Ask the AI" for deeper or what-if questions

8. Deployment Model

VHPCE is designed for progressive deployment across three tiers, each adding capability without requiring the student to redo previous steps:

Tier	What Students Get	Requirements
0 — Model	Full UI, every experiment (Model mode), interactive notebooks, <code>npm</code> + <code>pnpm</code> . Any OS.	NodeJS, >20.9
1 — Measured (CPU)	Real OpenMP/MPI runs on actual cores; Code Playground compiles and executes	Docker
2 — Measured (GPU)	CUDA experiments (occupancy, coalescing, divergence, warps) on actual GPUs	general-purpose container-toolkit

8.1 Zero-Install Option

The repository includes a devcontainer configuration for GitHub Codespaces. Students click "Code → Codespaces → Create codespace" and Tier 0 is ready immediately with all dependencies pre-installed. Tier 1 also works inside the Codespace via Docker-in-Docker.

8.2 Classroom Gateway

For institutional deployments, a single shared gateway gives an entire class Measured mode and a working Playground without requiring Docker on student machines. The instructor runs the gateway (Tiers 1/2) on a server and students point their web app at it via the `NEXT_PUBLIC_VHPCE_API` environment variable. The gateway's code runner is sandboxed (no network, capability-dropped, read-only filesystem, resource limits), but institutions are advised to place it behind authentication or a campus VPN.

8.3 Continuous Integration

A GitHub Actions workflow runs on every push and pull request: install, typecheck (TypeScript), lint (ESLint), and production build. This ensures the codebase stays clean across contributions.

9. Future Work

VHPCE has completed Phases 0 through 4 plus the Cloud Phase and engagement layer. Phase 5 is underway with the domain labs, and several directions remain for future development:

- **Additional domain modules.** FEM (heat/stress), CFD (Navier-Stokes lid-driven cavity), sparse solvers, FFT, and multigrid—each as a new tab in the Domain Labs hub, following the established pattern of tab-based navigation, rAF canvas animation, and domain-decomposition code snippets.
- **Measured backends for new experiments.** The seven new Flagship experiments (NUMA, Cache Hierarchy, SIMD, Tasks, Shared Memory, Collectives, Hybrid) are currently model-only. Adding gateway job kinds, kernels, and Docker images would bring them to full Model/Measured parity.
- **True hardware PMU counters.** WSL2 does not expose the host PMU, so cache-miss and IPC counters currently come from Valgrind cachegrind (simulated). Deploying on bare-metal Linux nodes would unlock `perf`/LIKWID/PAPI for precise hardware event counting.
- **Kubernetes autoscaling.** The Arq worker currently runs `max_jobs=1`. Moving sandbox execution to Kubernetes Jobs with a horizontal pod autoscaler would support concurrent multi-student workloads.
- **LMS integration.** Connecting the badge/streak system to institutional Learning Management Systems (Canvas, Moodle) for grade passback and assignment tracking.
- **Nsight Compute integration.** For GPU experiments, integrating Nsight Compute kernel-level counters (achieved occupancy, memory throughput, warp stall reasons) would add hardware-level detail beyond what the CUDA Occupancy API provides.

10. Conclusion

VHPCE demonstrates that the gap between HPC theory and practice can be effectively bridged by a platform that prioritizes **performance intuition over syntax memorization**. By building on a single data contract—the ProfileResult seam—the platform achieves a separation of concerns that is both architecturally clean and pedagogically powerful: students interact with the same charts, metrics, and explanations regardless of whether the data comes from an instant physics model or from a real measurement on their own hardware.

The predict-before-you-run methodology, the visual immediacy of animated hardware behavior, and the deterministic diagnostic engine work together to build the kind of intuition that is difficult to develop from textbooks and difficult to retain from lectures. The optional LLM layer extends this with conversational depth while remaining grounded in computed facts.

The tiered deployment model—from a zero-install Codespace to a GPU-accelerated Docker setup—ensures that the platform is accessible to students with varying hardware and institutional resources. The classroom gateway pattern lets instructors provide the full Measured experience without requiring Docker on every student machine.

With sixteen flagship experiments, a 214-entry command reference, four engineering domain labs, a sandboxed code playground, and a gamification layer, VHPCE provides a comprehensive environment for teaching parallel computing. The platform is open source under the GNU AGPL-3.0 license and available at <https://github.com/nikosvil/VHPCE>.